

Práctica Terraform Entrega

Terraform

▼ Práctica Terraform Entrega

▼ Terraform

Instalación y configuración básica

▼ Estructura básica de un proyecto

¿Qué es un proyecto en Terraform?

Ejemplo de `main.tf` básico

Archivo `variables.tf` : definición de entradas

Archivo `terraform.tfvars` : valores asignados

Archivo `outputs.tf` : resultados exportables

Separación de lógica por archivos

Organización recomendada por componente

Estructura de carpetas para múltiples entornos

Lenguaje HCL

¿Qué es HCL?

Estructura general de un bloque HCL

Tipos de datos en HCL

Comentarios en HCL

Interpolaciones y expresiones

▼ Operadores en HCL

Ejemplo completo en AWS con tipos y expresiones

▼ Buenas prácticas con HCL

Ejemplo de de arquitectura a desplegar



Nota

Recuerda que con `aws sts get-caller-identity` puedes verificar que estás utilizando el **rol de laboratorio** AWS.

Instalación y configuración básica

El repositorio para Amazon Linux 2023 sería...

<https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo>

Por tanto, los comandos para instalar Terraform en Amazon Linux 2023 serían:

```
sudo dnf install -y yum-utils
sudo yum-config-manager --add-repo https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
sudo dnf install -y terraform
```

En lugar de `sudo dnf install -y terraform` también podemos usar `sudo yum -y install terraform`, pero dnf es el gestor de paquetes recomendado para Amazon Linux 2023.

Estructura básica de un proyecto

¿Qué es un proyecto en Terraform?

Un proyecto en Terraform representa un conjunto de configuraciones organizadas para aprovisionar una infraestructura. El núcleo de un proyecto consiste en una o varias configuraciones `.tf` que definen recursos, variables, outputs, y proveedores.

Todo lo que se encuentra en una misma carpeta es interpretado por Terraform como un único módulo raíz. Estos archivos `.tf` se procesan en orden lógico, no alfabético ni por nombre de archivo, lo cual permite dividir la configuración en múltiples archivos sin afectar su funcionamiento.

Estructura general de configuración

Un proyecto típico suele tener esta estructura mínima:

```
📁 Proyecto
├─ main.tf – Recursos principales
├─ variables.tf – Declaración de variables
├─ outputs.tf – Outputs que expone el proyecto
└─ terraform.tfvars – Valores concretos para las variables
```

Cada archivo tiene un propósito específico, pero todos se combinan como una sola unidad de ejecución.

Guía de estilo de archivos

Ejemplo de `main.tf` básico

Un archivo `main.tf` define recursos y el proveedor. Ejemplo para AWS:

```
provider "aws" {  
  region = "us-west-2"  
}  
  
resource "aws_s3_bucket" "example" {  
  bucket = "mi-bucket-ejemplo"  
  acl    = "private"  
}
```

Este archivo puede contener uno o muchos recursos, o incluso incluir los bloques de variables directamente.

Ejemplo con AWS

Archivo `variables.tf` : definición de entradas

Las variables se declaran usando bloques `variable` . Ejemplo:

```
variable "region" {  
  description = "Región de despliegue"  
  type        = string  
  default     = "us-east-1"  
}  
  
variable "project_name" {  
  description = "Nombre del proyecto"  
  type        = string  
}
```

Estas variables pueden ser utilizadas dentro de `main.tf` con `var.nombre_variable` .

Variables en Terraform

Archivo `terraform.tfvars` : valores asignados

Puedes usar `terraform.tfvars` para definir los valores concretos que usarán las variables:

```
region      = "us-west-1"
project_name = "demo"
```

Terraform detecta este archivo automáticamente, y lo aplica al ejecutar `plan` o `apply`.

También puedes usar `*.auto.tfvars`, que siguen el mismo propósito pero permiten múltiples archivos.

■ Asignación de variables

Archivo `outputs.tf` : resultados exportables

Los outputs exponen información útil al final del `apply`, o para pasar valores entre módulos.

```
"resource_group_name" ">output "resource_group_name" {
  value      = azurerm_resource_group.rg.name
  description = "Nombre del resource group creado"
}
```

Pueden marcarse como `sensitive` si contienen datos sensibles que no deben mostrarse.

■ Outputs

Separación de lógica por archivos

Aunque se suelen usar archivos como `main.tf`, `variables.tf`, etc., en realidad no es obligatorio. Terraform procesa todos los archivos `.tf` juntos.

Puedes, por ejemplo, separar tu infraestructura así:

```
📁 Proyecto
├─ provider.tf – Configuración del proveedor
├─ networking.tf – Recursos de red (VPC, subredes...)
├─ compute.tf – Recursos de cómputo (instancias...)
└─ storage.tf – Recursos de almacenamiento
```

Esta organización ayuda a mantener limpio el proyecto conforme crece.

■ Convenciones de estructura

Organización recomendada por componente

Una recomendación habitual es agrupar por tipo de recurso:

- `networking.tf` : VPC, subnets, gateways
- `compute.tf` : EC2, Azure VMs
- `database.tf` : RDS, Azure SQL
- `security.tf` : IAM, NSG

Esto mejora la claridad y el mantenimiento del proyecto, especialmente en equipos grandes.

Estructura modular y escalable

Estructura de carpetas para múltiples entornos

Otra estructura habitual es separar por entorno:

```

📁 Proyecto
├── dev/
│   ├── main.tf – Recursos del entorno de desarrollo
│   └── terraform.tfvars – Variables para desarrollo
├── prod/
│   ├── main.tf – Recursos del entorno de producción
│   └── terraform.tfvars – Variables para producción
├── modules/
│   └── vpc/ – Módulo reutilizable de red (VPC)
```

Cada carpeta contiene una configuración idéntica pero con valores distintos, permitiendo despliegues paralelos por entorno.

Buenas prácticas de estructura por entorno

Lenguaje HCL

¿Qué es HCL?

HCL (HashiCorp Configuration Language) es el lenguaje de configuración utilizado por Terraform. Es un lenguaje **declarativo**, estructurado por bloques y diseñado para ser legible por humanos. Aunque tiene una sintaxis específica, también admite interpolaciones y expresiones lógicas.

El código en HCL suele estar organizado por **bloques**, con llaves y pares `clave = valor` . Un ejemplo típico sería un recurso `cloud`:

```
"aws_s3_bucket">resource "aws_s3_bucket" "ejemplo" {  
  bucket = "mi-bucket"  
  acl    = "private"  
}
```

Guía de sintaxis de configuración

Estructura general de un bloque HCL

Los bloques de HCL tienen esta forma:

```
<tipo> "<nombre_proveedor>" "<nombre_local>" {  
  argumento1 = valor  
  argumento2 = valor  
}
```

Por ejemplo:

```
"azurerm_resource_group">resource "azurerm_resource_group" "main" {  
  name      = "rg-ejemplo"  
  location = "westeurope"  
}
```

Los bloques pueden contener **atributos** (pares clave-valor), **bloques anidados** (como `tags` , `ingress` , etc.) y **metaargumentos** como `depends_on` , `count` , `for_each` .

Bloques y estructura HCL

Tipos de datos en HCL

HCL soporta varios tipos de datos básicos:

- **string:** "texto"
- **number:** 42 , 3.14
- **bool:** true / false
- **list:** ["a", "b", "c"]
- **map:** { key1 = "value1", key2 = "value2" }
- **tuple:** [true, 42, "hello"]
- **object:** { name = "Juan", edad = 30 }

Se pueden declarar tipos explícitamente en variables:

```
"regiones" ">variable "regiones" {
  type = list(string)
}
```

Tipos en Terraform

Comentarios en HCL

Puedes documentar tu código usando comentarios:

```
# Comentario de una línea

/*
Comentario de
varias líneas
*/
```

Es recomendable comentar bloques complejos o explicar decisiones de infraestructura para otros miembros del equipo o para mantenimiento futuro.

Comentarios en HCL

Interpolaciones y expresiones

Puedes referenciar otros valores usando la sintaxis `${...}` :

```
"aws_instance">resource "aws_instance" "web" {  
  tags = {  
    Name = "${var.entorno}-web"  
  }  
}
```

Desde Terraform 0.12, ya no es necesario `${}` en muchos casos. Puedes usar directamente:

```
Name = var.entorno
```

Interpolaciones y expresiones

Operadores en HCL

HCL incluye operadores lógicos y de comparación:

- Comparación: `==` , `!=` , `>` , `<` , `>=` , `<=`
- Booleanos: `&&` (and), `||` (or), `!` (not)
- Concatenación: `"prefix-${var.nombre}"`

Ejemplo:

```
locals {  
  es_produccion = var.env == "prod"  
}
```

Expresiones condicionales

Ejemplo completo en AWS con tipos y expresiones

```
"env" ">variable "env" {  
  type    = string  
  default = "dev"  
}  
  
"aws_s3_bucket">resource "aws_s3_bucket" "logs" {  
  bucket = "${var.env}-logs"  
  acl    = "private"  
  
  tags = {  
    Environment = var.env  
  }  
}
```

Este ejemplo muestra uso de variables, interpolaciones, tipos y etiquetas.

■ Ejemplo con variables y recursos

Buenas prácticas con HCL

- Usa nombres significativos para recursos y variables.
- Documenta tu código con comentarios útiles.
- Separa los archivos por propósito (`main.tf` , `variables.tf` , `outputs.tf`).
- Usa tipos explícitos para evitar errores.
- Valida tu configuración con `terraform validate` .

Estas prácticas ayudan a mantener proyectos legibles, predecibles y colaborativos.

■ Guía de estilo y validación

Ejemplo de de arquitectura a desplegar

Se propone para la arquitectura un diagrama similar al siguiente:

@startuml VPC

' <https://awslabs.github.io/aws-icons-for-plantuml/>

!define AWSPuml <https://raw.githubusercontent.com/awslabs/aws-icons-for-plantuml/v23.0/dist>

```
!include AWSPuml/AWSCommon.puml
!include AWSPuml/AWSSimplified.puml
!include AWSPuml/Groups/AWSCloud.puml
!include AWSPuml/Groups/VPC.puml
!include AWSPuml/Groups/AvailabilityZone.puml
!include AWSPuml/Groups/PublicSubnet.puml
!include AWSPuml/Groups/PrivateSubnet.puml
!include AWSPuml/Groups/AutoScalingGroup.puml
!include AWSPuml/NetworkingContentDelivery/VPCNATGateway.puml
!include AWSPuml/NetworkingContentDelivery/VPCInternetGateway.puml
!include AWSPuml/NetworkingContentDelivery/ElasticLoadBalancingApplicationLoadBalancer.puml
!include AWSPuml/Database/AuroraMySQLInstance.puml
!include AWSPuml/Compute/EC2Instance.puml
!include AWSPuml/Storage/EFS.puml
!include AWSPuml/ManagementGovernance/CloudFormationTemplate.puml
```

hide stereotype

scale 600 width

left to right direction

' skinparam linetype ortho

AWSCloudGroup(cloud) {

 VPCInternetGateway(igw, "Internet\nGateway", "")

 VPCGroup(vpc) {

 ElasticLoadBalancingApplicationLoadBalancer(alb, "Balanceador\nde carga (ALB)", "")

 AvailabilityZoneGroup(az_1a, "AZ us-east-1a") {

 AutoScalingGroupGroup(asg_a, "AutoScaling") #transparent {

 PublicSubnetGroup(pub_a, "Subred pública\nren zona a") #technology {

 VPCNATGateway(nat_gateway_a, "NAT\nGateway", "") #transparent

 }

 PrivateSubnetGroup(priv_a, "Subred privada\nren zona a") #azure {

```

        AuroraMySQLInstance(rds_a, "RDS\nMySQL", "") #transparent
        EC2Instance(ec2_a, "Instancia T3\ncon\nWordpress", "") #transparent
        rds_a <-d-> ec2_a
    }
}

AvailabilityZoneGroup(az_1b, "AZ us-east-1b") {
    PublicSubnetGroup(pub_b, "Subred pública\nren zona b") #technology {
        VPCNATGateway(nat_gateway_b, "NAT\nGateway", "") #transparent
    }
    PrivateSubnetGroup(priv_b, "Subred privada\nr en zona b") #azure {
        AuroraMySQLInstance(rds_b, "RDS\nMySQL", "") #transparent
        EC2Instance(ec2_b, "Instancia T3\ncon\nWordpress", "") #transparent
        rds_b <-d-> ec2_b
    }
}

}

EFS(efs, "Elastic File\nSystem", "")
CloudFormationTemplate(launchTemplate, "Launch Template\ncon user data", "")

igw <-u-> alb
igw <-l- nat_gateway_a
igw <-l- nat_gateway_b
alb <-u-> ec2_a
alb <-u-> ec2_b
nat_gateway_a <-u- ec2_a
nat_gateway_b <-u- ec2_b

launchTemplate -d-> asg_a
efs <.d.> ec2_a
efs <.d.> ec2_b
}

@enduml

```